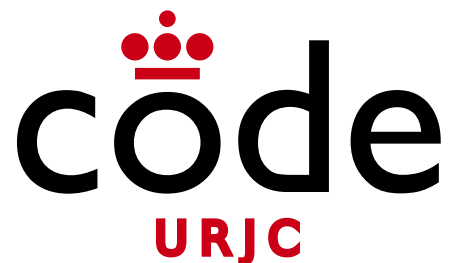


Fundamentos de la Web

Bloque III: Tecnologías de servidor web

Tema 3.5: Módulos en Node



©2024

Micael Gallego, Francisco Gortázar, Michel Maes, Óscar Soto, Iván Chicano

Algunos derechos reservados

Este documento se distribuye bajo la licencia
“Atribución-CompartirIgual 4.0 Internacional”
de Creative Commons Disponible en
<https://creativecommons.org/licenses/by-sa/4.0/deed.es>

Librerías incluidas en Node

- Node ofrece funcionalidad que se utiliza mediante **módulos**
- Al importar un módulo de Node se pone el prefijo 'node:' y no se pone extensión

```
import fs from 'node:fs';
```

- Alguna funcionalidad está disponible sin tener que importar su módulo de forma explícita

```
console.log('Hello World');
```

Librerías incluidas en Node

- Lectura de un fichero en Node con el módulo **File System** (fs)
 - Usando el **default export**

```
import fs from 'node:fs';

fs.readFile('/home/data.txt', 'utf8', (err, contents) => {
  if (err) {
    return console.error(err);
  }
  console.log(contents);
});
```

Librerías incluidas en Node

- Lectura de un fichero en Node con el módulo **File System** (fs)
 - Usando la función directamente (**named export**)

```
import { readFile } from 'node:fs';

readFile('/home/data.txt', 'utf8', (err, contents) => {
  if (err) {
    return console.error(err);
  }
  console.log(contents);
});
```

Librerías incluidas en Node

- Algunos módulos incluidos en Node.js

Módulo	Propósito / funcionalidad principal
fs (File System)	Operaciones de lectura, escritura, directorios, streams de archivos, permisos, etc.
path	Construcción, normalización, resolución de rutas de sistemas de archivos.
http / https / http2	Crear servidores y clientes HTTP, HTTPS, soporte HTTP/2
events	Emisión y escucha de eventos (EventEmitter)
stream	Abstracción de flujos de datos leíbles, escribibles o dúplex.
buffer	Representación de datos binarios (arrays de bytes)
crypto	Funciones criptográficas: hashes, cifrado, generación de claves, firmas
process	Información y control del proceso Node.js: variables de entorno, señalización, ciclo de vida
worker_threads	Hilos con ejecución paralela de JavaScript
assert	Aserciones para pruebas / validaciones
timers	setTimeout, setInterval, y sus versiones basadas en promesas
URL / querystring	Análisis y formateo de URLs y cadenas de consulta

<https://nodejs.org/api/documentation.html>

Librerías externas

- **Node Package Manager (NPM):** Herramienta que permite **descargar** módulos Node
- Existe un **repositorio público** con módulos **software libre**
- También se pueden configurar **repositorios privados** para **módulos privados**



<https://npmjs.com>

Node Package Manager (NPM)

- Una aplicación define los paquetes que necesita en un fichero llamado **package.json**
- Este fichero suele tener el nombre de la aplicación y la versión

```
{
  "name": "my-app",
  "version": "1.0.0",
  "type": "module"
}
```

- Se puede crear con el comando

```
$ npm init
```

Node Package Manager (NPM)

- Los paquetes de los que se depende se especifican en la sección "dependencies"

```
{
  "name": "my-app",
  "version": "1.0.0",
  "type": "module",
  "dependencies": {
    "lodash": "4.17.15"
  }
}
```

- Para instalar el/los paquete/s

```
$ npm install
```

Node Package Manager (NPM)

- Una vez instalado el paquete, ya se puede importar en el código del proyecto

```
// El paquete lodash exporta un objeto default
import lodash from 'lodash';

let output = lodash.without([1, 2, 3], 1);

console.log(output);
```

Node Package Manager (NPM)

- Los paquetes descargados se guardan en la carpeta **node_modules** del proyecto
- Esta carpeta se debe incluir en el **.gitignore** porque los paquetes no deben subirse al repositorio de código (ya que se pueden descargar en cualquier momento desde la red con **npm install**)

Node Package Manager (NPM)

- Se puede descargar una dependencia en su última versión y añadirla al `package.json` con el comando

```
$ npm install --save <lib>
```

- Por ejemplo la librería `left-pad`

```
$ npm install --save left-pad
```

Node Package Manager (NPM)

- Se pueden instalar paquetes con herramientas de desarrollo, no para ser usados desde el código de la aplicación (nodemon, angular-cli...)
- Estos paquete se especifican en la sección **"devDependencies"** del `package.json`

```
{
  "name": "my-app",
  "version": "1.0.0",
  "type": "module",
  "dependencies": {
    "lodash": "4.17.15"
  },
  "devDependencies": {
    "nodemon": "3.1.4"
  }
}
```

Node Package Manager (NPM)

- Es habitual que algunas herramientas se instalen globalmente en el sistema, no de forma concreta en un proyecto (en linux necesitan **sudo**)

```
$ npm install -g nodemon
```

- Estas herramientas se registran de forma automática en el **PATH** del sistema y se pueden ejecutar en cualquier carpeta

```
$ nodemon app.js
```

Módulos CommonsJS

- Cuando se desarrolló Node, el sistema de **módulos estándar (ESM)** no estaba diseñado
- Node diseñó su propio sistema: **CommonJS**
- Se recomienda usar el sistema estándar (ESM), pero existe mucha **documentación y ejemplos** que todavía usan el sistema antiguo

- Se puede **convertir** entre ellos de forma automática (salvo algunos casos especiales)

Tarea	CommonJS	ESM
Importar módulo	<code>const x = require('x');</code>	<code>import x from 'x';</code>
Importar elemento	<code>const { a } = require('x');</code>	<code>import { a } from 'x';</code>
Exportar único valor	<code>module.exports = x;</code>	<code>export default x;</code>
Exportar varios	<code>exports.a = 1;</code>	<code>export const a = 1;</code>
Importar fichero	<code>require('./file');</code>	<code>import './file.js';</code>

VSCode sugiere la conversión de `require` a `import` de forma automática

Ejercicio 1

- Reimplementa el ejercicio de arrays para usar **lodash** para eliminar los ceros de los arrays
- Busca en la documentación de lodash qué método o métodos podrían serte útiles para esta tarea
- <https://lodash.com/docs>

